Final Project Report

Explainable Network Intrusion Detection Using Machine Learning

By: Damini

Table of Contents

Abstract 3

1. Problem Definition and Project Goals 3

2. Dataset Description 3

   2.1 Dataset Variables 4

3. Related Work 7

4. Data Exploration and Preprocessing 7

   4.1 Loading and Cleaning 7

   4.2 Class Distribution 8

   4.3 Feature Distributions and Correlation 9

5. Data Analysis and Experimental Results 12

   5.1 Modeling Setup 12

   5.2 Hyperparameter Tuning 12

   5.3 Model Evaluation 12

   5.4 Benchmarking and Best Model 14

6. Explainability with LIME 15

   6.1 Attack Flow Explanation 15

   6.2 Benign Flow Explanation 17

7. Fairness Audit 18

8. Conclusion and Future Work 18

References 18

Contributions 19

Abstract

This project uses machine learning to detect malicious network traffic from the CIC-IDS2017 dataset. The task was changed into a binary classification problem where each network flow is classified as either benign or attack traffic. The data was loaded in memory-efficient chunks, cleaned, sampled, checked for missing and infinite values, and then split into training and test sets. Three supervised models were trained and compared: Logistic Regression, Random Forest, and XGBoost. The models were evaluated using accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, ROC curves, and precision-recall curves. XGBoost produced the best overall result with 0.9991 accuracy, 0.9955 precision, 0.9995 recall, 0.9975 F1-score, and 1.0000 ROC-AUC on the test set. LIME was used to explain individual model predictions so that the model output could be connected to actual network-flow features. Overall, the project shows that machine learning can classify sampled CIC-IDS2017 traffic very accurately, while explanation tools can make the results easier to understand for security analysis.

1. Problem Definition and Project Goals

Network traffic is too large and fast moving to inspect manually. Traditional rule based intrusion detection systems can be useful, but they may miss new or changing attack behavior if a matching rule does not already exist. The purpose of this project is to build a machine learning model that can look at flow level network features and classify a flow as benign or malicious. The project also adds explainability so that a prediction is not treated as a black-box result.

The main project goals were:

- Load and clean the CIC-IDS2017 network intrusion detection dataset.
- Convert the original attack labels into a binary label: Benign = 0 and Attack = 1.
- Explore the class distribution and the behavior of selected network-flow features.
- Train and tune Logistic Regression, Random Forest, and XGBoost models.
- Compare the models using classification metrics that are useful for intrusion detection.
- Use LIME to explain one attack prediction and one benign prediction from the best model.
- Summarize the strengths, limitations, and possible future improvements.

2. Dataset Description

The dataset used in this project is CIC-IDS2017 from the Canadian Institute for Cybersecurity at the University of New Brunswick. It is a public intrusion detection dataset that includes benign traffic and multiple attack types. The dataset is available in packet capture form and also as machine-learning CSV files containing labeled network flows [1]. In the final notebook, the dataset was loaded from eight CSV files representing different weekday capture sessions and attack scenarios.

The original dataset contains several attack categories, including DoS, DDoS, PortScan, Botnet, FTP-Patator, SSH-Patator, Web Attacks, Infiltration, and Heartbleed. For this project, the original labels were simplified into a binary problem. BENIGN was encoded as 0, and every attack type was encoded as 1.

Dataset Item	Value Used in Project
Dataset name	CIC-IDS2017
Source	Canadian Institute for Cybersecurity, University of New Brunswick
Files loaded	8 CSV files
Task type	Binary classification
Target variable	binary_label
Target classes	0 = Benign, 1 = Attack
Predictor features	78 numeric network-flow features
Examples of features	Flow Duration, Total Fwd Packets, Total Backward Packets, Flow Bytes/s, Flow Packets/s, SYN Flag Count, ACK Flag Count, Packet Length statistics

2.1 Dataset Variables

In the final processed dataset, the non-numeric identifier columns such as Flow ID, Source IP, Destination IP, and Timestamp were removed because they were not used for model training. The remaining dataset contains network-flow features, the original attack label, and the binary target label.

Variable Name	Data Type	Explanation
Destination Port	int64	Destination port number used by the network flow.
Flow Duration	int64	Total duration of the network flow.
Total Fwd Packets	int64	Total number of packets sent in the forward direction.
Total Backward Packets	int64	Total number of packets sent in the backward direction.
Total Length of Fwd Packets	int64	Total size of all packets sent in the forward direction.
Total Length of Bwd Packets	int64	Total size of all packets sent in the backward direction.
Fwd Packet Length Max	int64	Maximum packet length in the forward direction.
Fwd Packet Length Min	int64	Minimum packet length in the forward direction.
Fwd Packet Length Mean	float32	Average packet length in the forward direction.
Fwd Packet Length Std	float32	Standard deviation of packet length in the forward direction.
Bwd Packet Length Max	int64	Maximum packet length in the backward direction.
Bwd Packet Length Min	int64	Minimum packet length in the backward direction.
Bwd Packet Length Mean	float32	Average packet length in the backward direction.
Bwd Packet Length Std	float32	Standard deviation of packet length in the backward direction.
Flow Bytes/s	float32	Number of bytes transferred per second in the flow.
Flow Packets/s	float32	Number of packets transferred per second in the flow.
Flow IAT Mean	float32	Average inter-arrival time between packets in the flow.
Flow IAT Std	float32	Standard deviation of packet inter-arrival time in the flow.
Flow IAT Max	int64	Maximum inter-arrival time between packets in the flow.
Flow IAT Min	int64	Minimum inter-arrival time between packets in the flow.

Fwd IAT Total	int64	Total inter-arrival time for forward packets.
Fwd IAT Mean	float32	Average inter-arrival time for forward packets.
Fwd IAT Std	float32	Standard deviation of inter-arrival time for forward packets.
Fwd IAT Max	int64	Maximum inter-arrival time for forward packets.
Fwd IAT Min	int64	Minimum inter-arrival time for forward packets.
Bwd IAT Total	int64	Total inter-arrival time for backward packets.
Bwd IAT Mean	float32	Average inter-arrival time for backward packets.
Bwd IAT Std	float32	Standard deviation of inter-arrival time for backward packets.
Bwd IAT Max	int64	Maximum inter-arrival time for backward packets.
Bwd IAT Min	int64	Minimum inter-arrival time for backward packets.
Fwd PSH Flags	int64	Number of PSH flags in the forward direction.
Bwd PSH Flags	int64	Number of PSH flags in the backward direction.
Fwd URG Flags	int64	Number of URG flags in the forward direction.
Bwd URG Flags	int64	Number of URG flags in the backward direction.
Fwd Header Length	int64	Total header length of forward packets.
Bwd Header Length	int64	Total header length of backward packets.
Fwd Packets/s	float32	Number of forward packets per second.
Bwd Packets/s	float32	Number of backward packets per second.
Min Packet Length	int64	Minimum packet length in the flow.
Max Packet Length	int64	Maximum packet length in the flow.
Packet Length Mean	float32	Average packet length in the flow.
Packet Length Std	float32	Standard deviation of packet length in the flow.
Packet Length Variance	float32	Variance of packet length in the flow.
FIN Flag Count	int64	Count of FIN flags in the flow.
SYN Flag Count	int64	Count of SYN flags in the flow.
RST Flag Count	int64	Count of RST flags in the flow.
PSH Flag Count	int64	Count of PSH flags in the flow.
ACK Flag Count	int64	Count of ACK flags in the flow.

URG Flag Count	int64	Count of URG flags in the flow.
CWE Flag Count	int64	Count of CWE flags in the flow.
ECE Flag Count	int64	Count of ECE flags in the flow.
Down/Up Ratio	int64	Ratio between download and upload traffic.
Average Packet Size	float32	Average size of packets in the flow.
Avg Fwd Segment Size	float32	Average segment size in the forward direction.
Avg Bwd Segment Size	float32	Average segment size in the backward direction.
Fwd Header Length.1	int64	Duplicate forward header length feature kept from the original dataset.
Fwd Avg Bytes/Bulk	int64	Average number of bytes per bulk transfer in the forward direction.
Fwd Avg Packets/Bulk	int64	Average number of packets per bulk transfer in the forward direction.
Fwd Avg Bulk Rate	int64	Average bulk transfer rate in the forward direction.
Bwd Avg Bytes/Bulk	int64	Average number of bytes per bulk transfer in the backward direction.
Bwd Avg Packets/Bulk	int64	Average number of packets per bulk transfer in the backward direction.
Bwd Avg Bulk Rate	int64	Average bulk transfer rate in the backward direction.
Subflow Fwd Packets	int64	Number of packets in the forward subflow.
Subflow Fwd Bytes	int64	Number of bytes in the forward subflow.
Subflow Bwd Packets	int64	Number of packets in the backward subflow.
Subflow Bwd Bytes	int64	Number of bytes in the backward subflow.
Init_Win_bytes_forward	int64	Initial TCP window size in the forward direction.
Init_Win_bytes_backward	int64	Initial TCP window size in the backward direction.
act_data_pkt_fwd	int64	Number of forward packets containing actual data.
min_seg_size_forward	int64	Minimum segment size in the forward direction.
Active Mean	float32	Average time the flow was active before becoming idle.
Active Std	float32	Standard deviation of active time.
Active Max	int64	Maximum active time in the flow.
Active Min	int64	Minimum active time in the flow.
Idle Mean	float32	Average time the flow stayed idle.
Idle Std	float32	Standard deviation of idle time.

Idle Max	int64	Maximum idle time in the flow.
Idle Min	int64	Minimum idle time in the flow.
Label	object	Original class label from the dataset, such as BENIGN, DoS Hulk, DDoS, PortScan, or Web Attack. This was kept for exploration.
binary_label	int64	Final target variable created for this project. 0 represents benign traffic and 1 represents attack traffic.

The final modeling task used binary_label as the target variable. The original Label column was kept for exploratory analysis so that the distribution of different attack categories could still be reviewed. All other variables are numerical network-flow features extracted from packet traffic. These features describe packet counts, packet sizes, timing behavior, TCP flags, header lengths, subflow information, and active/idle behavior of each network flow.

3. Related Work

- Sharafaldin, Lashkari, and Ghorbani created the CIC-IDS2017 dataset to support realistic intrusion detection research. Their work includes benign traffic and common attacks such as DoS, DDoS, brute force, web attacks, infiltration, port scan, and botnet. The dataset also provides more than 80 extracted network-flow features, which makes it useful for machine learning classification.
- CICFlowMeter is also related to this project because it is the tool used to convert packet capture traffic into flow-based features. It extracts features such as packet counts, flow duration, byte counts, and forward/backward traffic statistics. These are the same types of features used in this project.
- Recent work has also explored newer intrusion detection methods such as contrastive self-supervised learning. One example is CLAN, which tries to improve intrusion detection when large labeled datasets are not always available. This project uses a simpler supervised approach by comparing Logistic Regression, Random Forest, and XGBoost for binary attack detection.

4. Data Exploration and Preprocessing

4.1 Loading and Cleaning

The full CIC-IDS2017 dataset is large, so a memory-efficient approach was used in Google Colab. Each CSV file was read in chunks of 50,000 rows. During loading, unnecessary columns were removed, infinite values were replaced with missing values, rows with missing values were dropped, and float columns were converted from float64 to float32 to reduce memory use. A 10% sample was taken from each chunk before combining all chunks into one DataFrame. This avoided RAM crashes while still keeping traffic from every file.

Preprocessing Step	Result
Files read	8 CSV files
Chunk size	50,000 rows
Sampling method	10% sample from each chunk
Rows after initial chunked loading	282,784
Columns after initial loading	80
Duplicate rows found	14,736
Rows after duplicate removal	268,048
Missing values after cleaning	0
Infinite values after cleaning	0
Predictor matrix shape	268,048 rows x 78 features

The columns Flow ID, Source IP, Destination IP, and Timestamp were removed because the goal was to classify traffic from flow behavior instead of using identifiers. After cleaning, the dataset contained 78 numeric features plus the original label and binary label. The numeric features were used for modeling. Below is the Target variable distribution.

Class	Count	Percentage
Benign (0)	219,486	81.88%
Attack (1)	48,562	18.12%

4.2 Class Distribution

The multi-class label plot shows that benign traffic is the largest class, while the attack classes are much smaller. The most frequent attack groups in the sampled data are DoS Hulk, PortScan, and DDoS. The binary label plot also shows class imbalance: benign traffic makes up most of the dataset.

Label	
BENIGN	219486
DoS Hulk	17773
PortScan	14444
DDoS	12800
DoS GoldenEye	981
FTP-Patator	674
DoS slowloris	556
DoS Slowhttptest	530
SSH-Patator	406
Bot	183
Web Attack &#65533; Brute Force	150
Web Attack &#65533; XSS	59
Infiltration	3
Web Attack &#65533; Sql Injection	2
Heartbleed	1

Figure 1. Distribution of original multi-class labels.

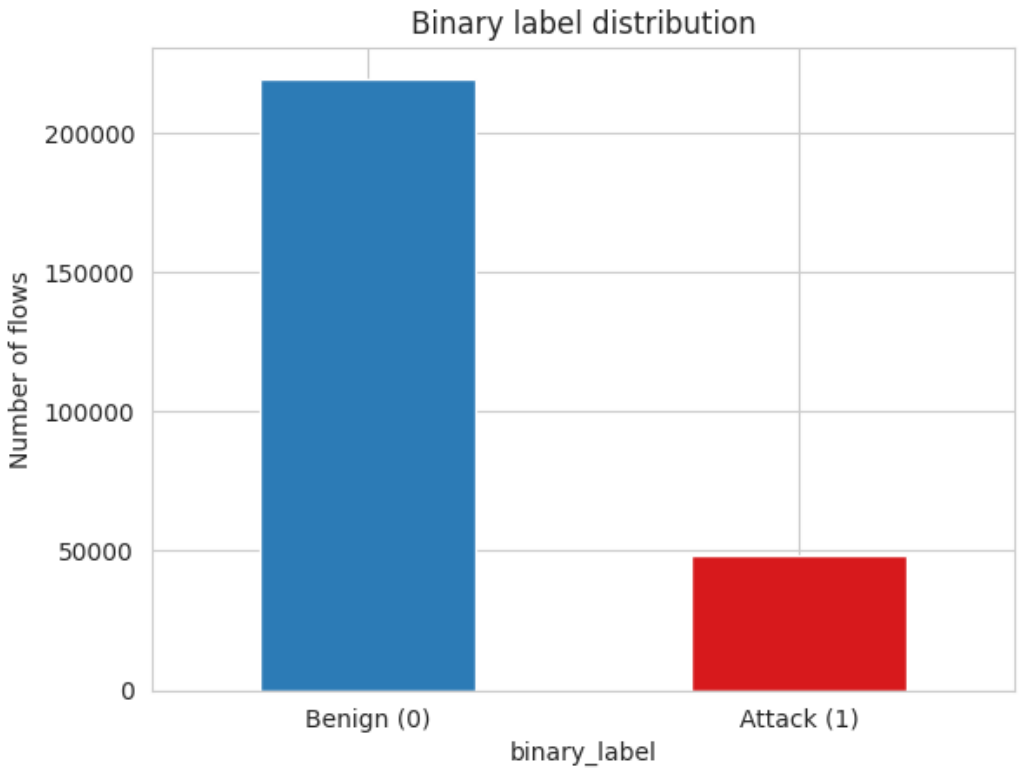


Figure 2. Binary class distribution after label encoding.

4.3 Feature Distributions and Correlation

The EDA plot shows that several network-flow features are highly skewed. Flow Duration has many small values and some very large values. Packet count features such as Total Fwd Packets and Total Backward Packets also contain outliers. This is expected in network traffic because most flows are short, while a smaller number of flows can last longer or transfer many more packets.

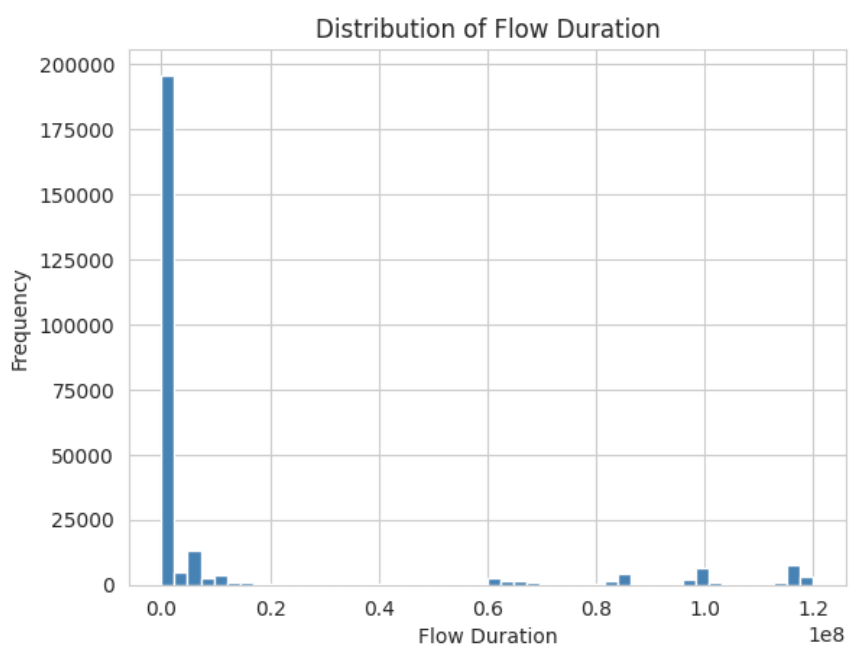


Figure 3. Histogram of Flow Duration.

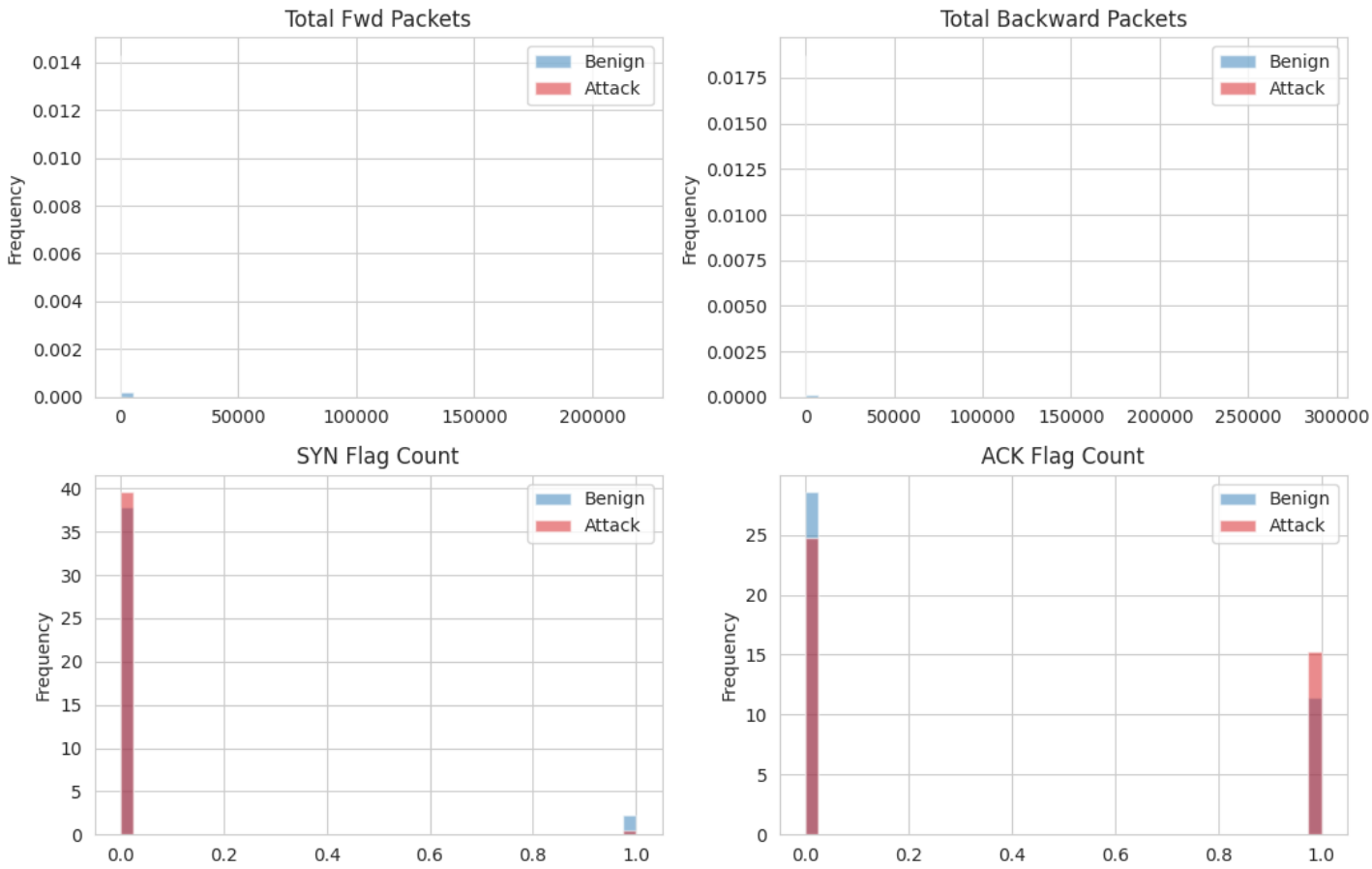
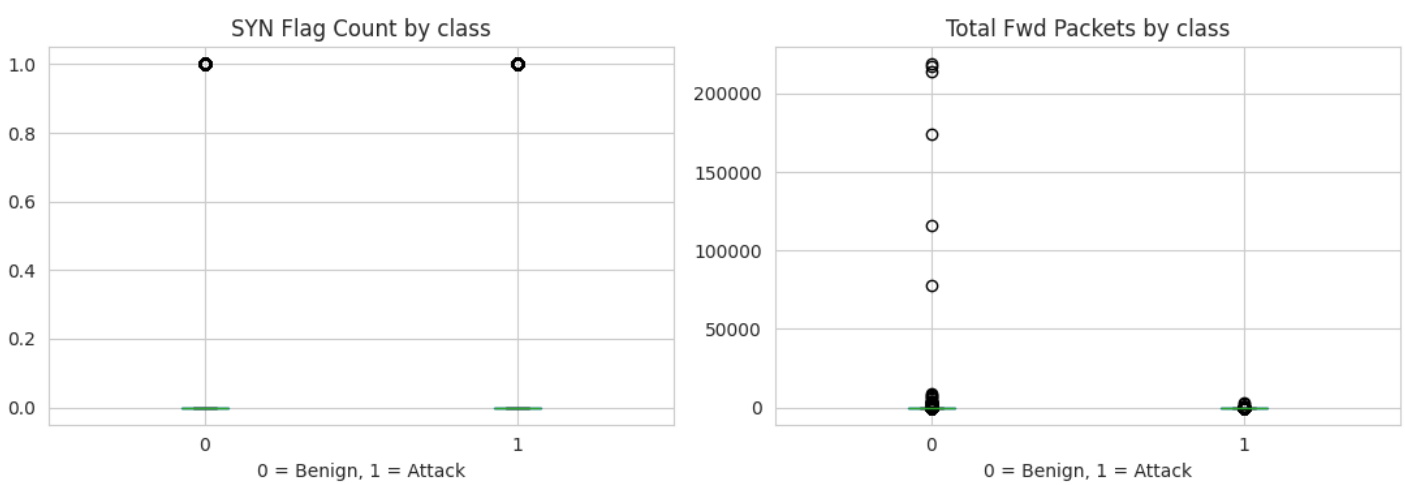


Figure 4. Histograms of selected features split by class.

The boxplots show that SYN Flag Count is mostly near zero for both benign and attack flows. Total Fwd Packets contains extreme outliers, especially in benign traffic. These plots show that a single feature may not fully separate benign and attack traffic, so the models need to combine many features together.



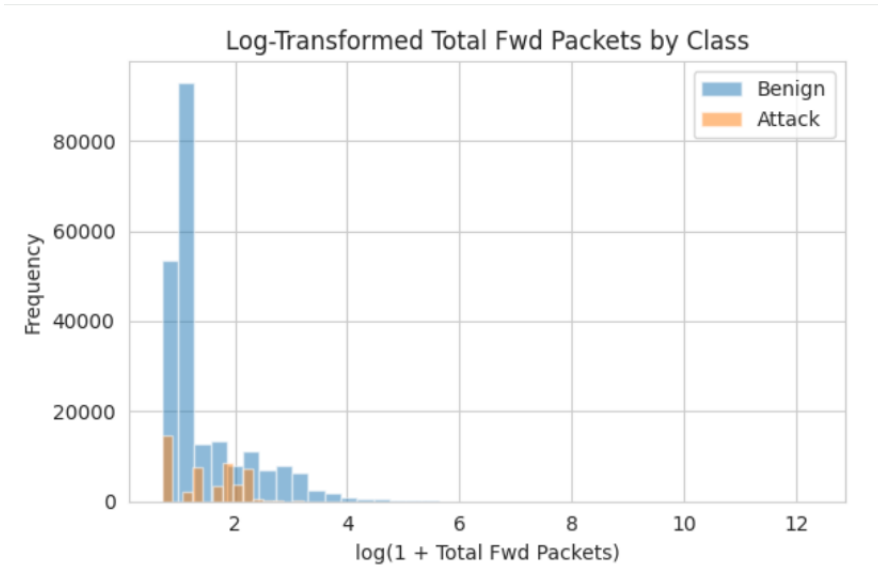


Figure 5. Boxplots for SYN Flag Count and Total Fwd Packets by class.

The correlation heatmap was created using selected traffic features and the binary label. The values show that some features have visible relationships with the label, but no single feature fully explains the target. This supports the use of machine learning models that can combine multiple network-flow patterns.

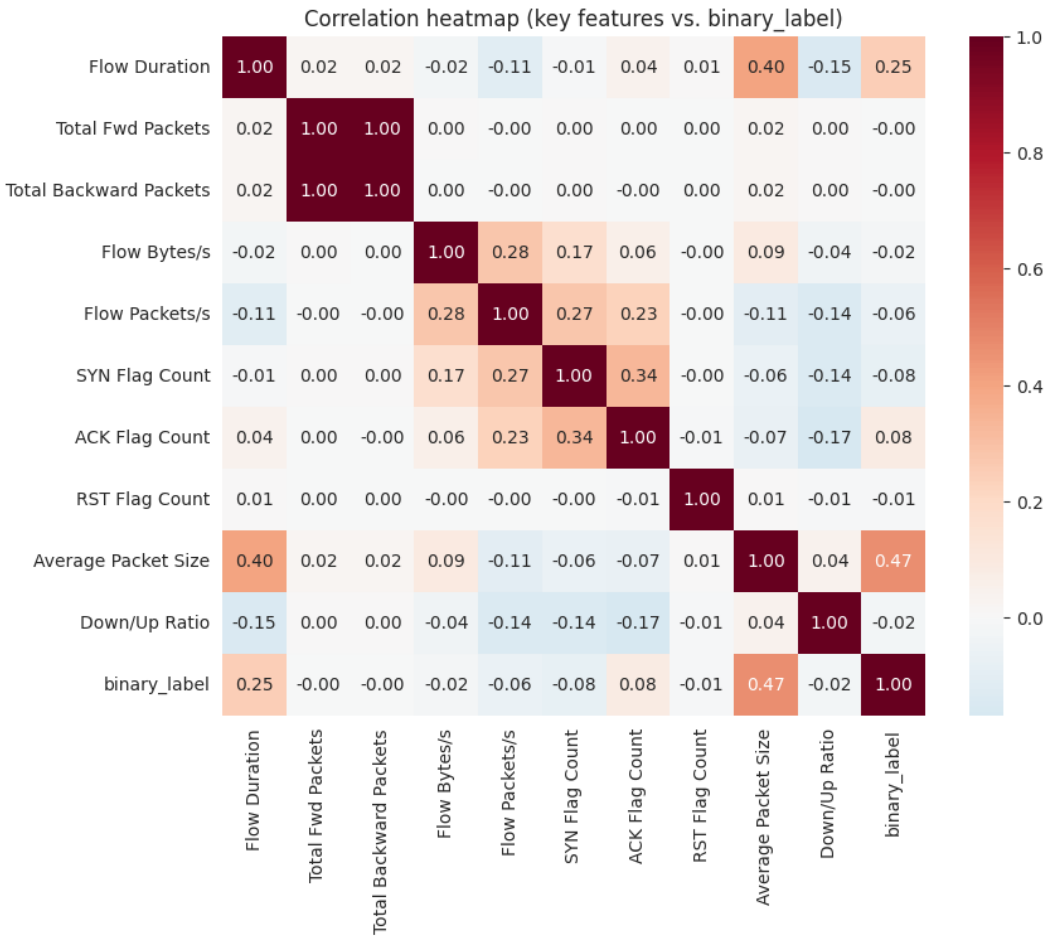


Figure 6. Correlation heatmap for selected features and binary label.

5. Data Analysis and Experimental Results

5.1 Modeling Setup

After cleaning, the original Label column and binary_label column were separated from the predictors. The final feature matrix had 268,048 rows and 78 predictor features. The target vector contained the binary labels. The data was split into training and test sets using an 80/20 stratified split. Stratification kept the class ratio similar in both sets.

Split	Rows	Columns / Target
Training set	214,438	78 predictor features
Test set	53,610	78 predictor features
Test benign support	43,898	Class 0
Test attack support	9,712	Class 1

StandardScaler was applied to Logistic Regression because it is sensitive to feature scale. Tree-based models such as Random Forest and XGBoost were trained on the unscaled features because decision trees do not require feature scaling in the same way.

5.2 Hyperparameter Tuning

Hyperparameter tuning was performed on a 30,000-row sample from the training set to reduce compute time. The sample was selected from the training data only, so the test set remained untouched until final evaluation. GridSearchCV with 3-fold stratified cross-validation was used, and F1-score was selected as the tuning metric because it balances precision and recall.

Model	Parameters Tried	Best Parameters	Best CV F1-score
Logistic Regression	C: 0.01, 0.1, 1.0, 10.0	C = 10.0	0.8263
Random Forest	"n_estimators": [100, 200], "max_depth": [10, 20, None], "min_samples_split": [2, 5], "min_samples_leaf": [1, 2]	max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 200	0.9923
XGBoost	"n_estimators": [100, 200], "max_depth": [4, 6, 8], "learning_rate": [0.05, 0.1, 0.2], "subsample": [0.8, 1.0], "colsample_bytree": [0.8, 1.0]	‘colsample_bytree': 1.0, 'learning_rate': 0.2, 'max_depth': 4, 'n_estimators': 200, 'subsample': 0.8	0.9954

5.3 Model Evaluation

The final tuned models were trained on the full training set and evaluated on the held-out test set. For intrusion detection, recall is especially important because a false negative means an attack was missed. Precision is also important because too many false positives can overload analysts. F1-score was used as the main balance between these two metrics.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.9331	0.7367	0.9819	0.8418	0.9859
Random Forest	0.9988	0.9959	0.9975	0.9967	0.9999
XGBoost	0.9991	0.9955	0.9995	0.9975	1.0000

Model	True Benign	Benign Predicted as Attack	Attack Predicted as Benign	True Attack
Logistic Regression	40,490	3,408	176	9,536
Random Forest	43,858	40	24	9,688
XGBoost	43,854	44	5	9,707

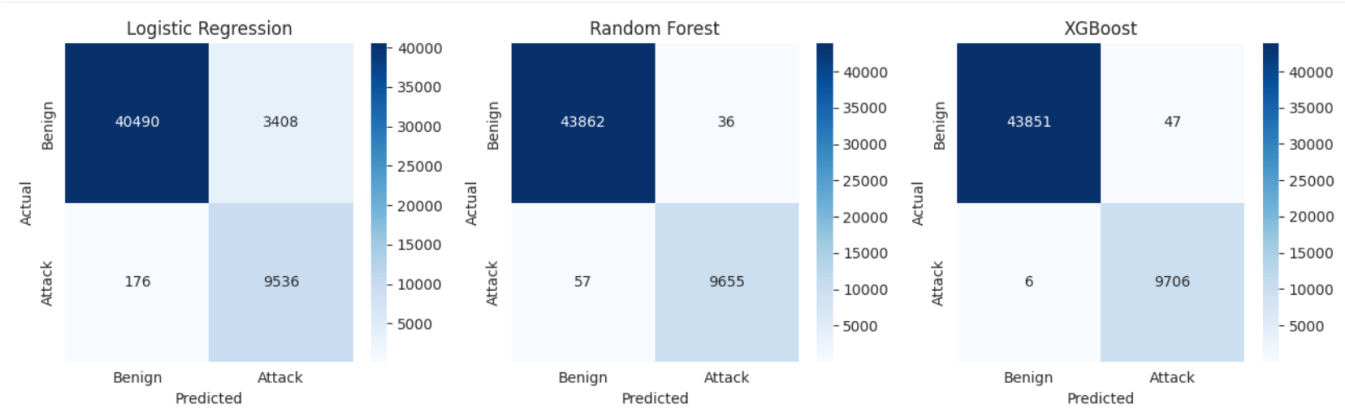


Figure 7. Confusion matrices for all three models.

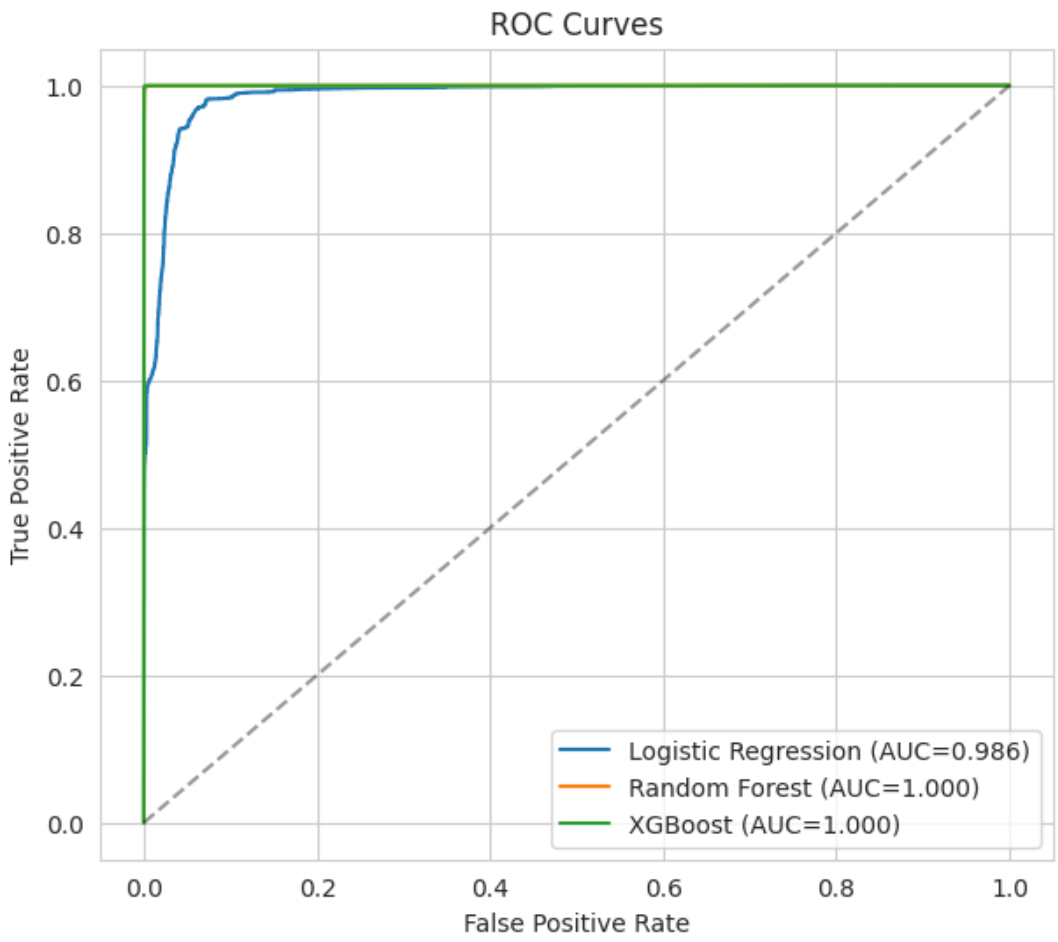


Figure 8. ROC curves for Logistic Regression, Random Forest, and XGBoost.

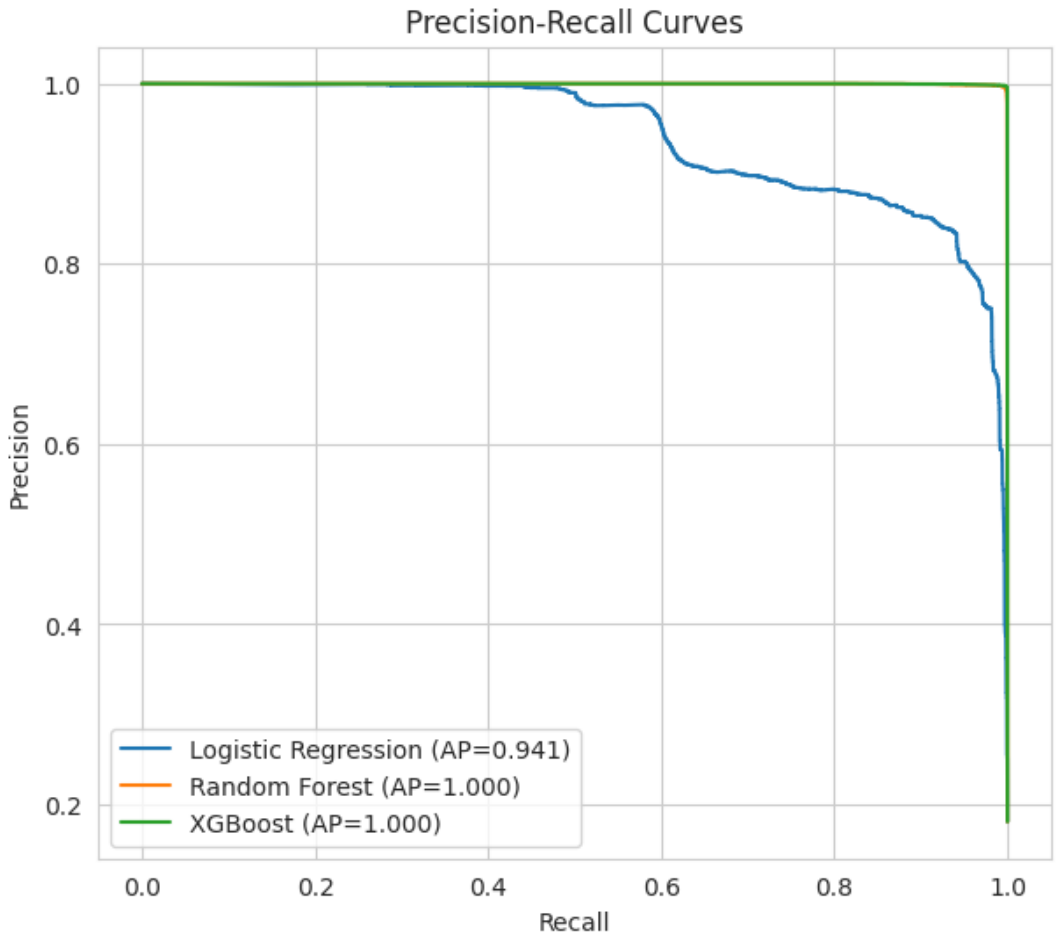


Figure 9. Precision-recall curves for Logistic Regression, Random Forest, and XGBoost.

5.4 Benchmarking and Best Model

The baseline Logistic Regression model had strong recall, but its precision was much lower than the ensemble models. This means it detected most attacks, but it also produced many more false positives. Random Forest performed much better and reduced both false positives and false negatives. XGBoost gave the best overall result, with the highest recall and F1-score and only five attack flows incorrectly predicted as benign in the test set.

XGBoost was selected as the best model because it had the strongest balance of precision, recall, and F1-score. For intrusion detection, its very low false-negative count is important because missed attacks are more dangerous than extra alerts. However, the very high score should still be interpreted carefully because the experiment used a sampled version of CIC-IDS2017. A stronger final validation would test the model on a separate dataset or on newer traffic collected from a different network.

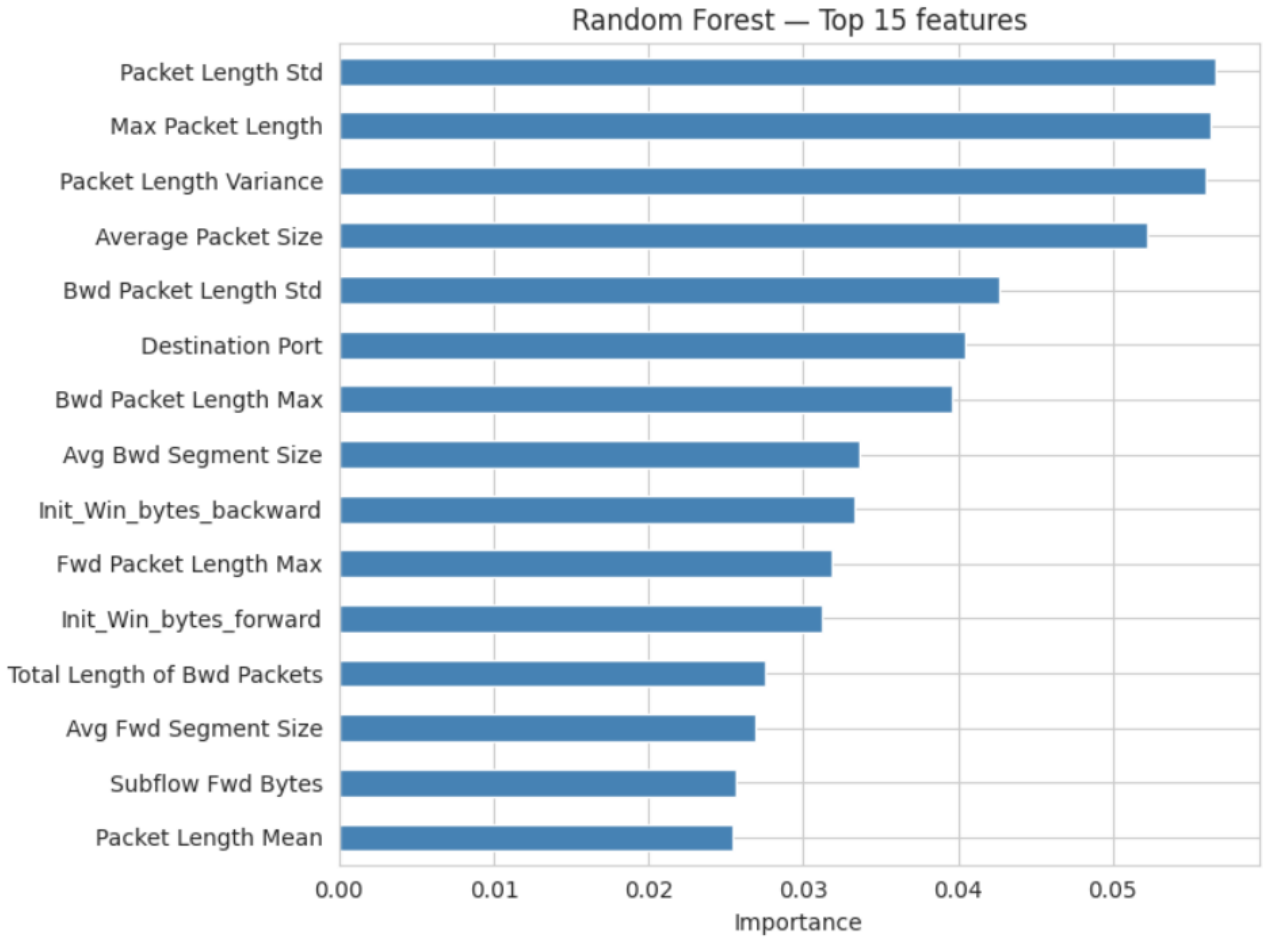


Figure 10. Random Forest top 15 feature importances.

The Random Forest feature importance plot shows that packet length and packet size statistics were among the most useful features. Examples include Max Packet Length, Packet Length Std, Average Packet Size, and Bwd Packet Length Std. This makes sense because attack traffic can change packet sizes, flow patterns, and timing behavior compared with normal traffic.

6. Explainability with LIME

The final notebook used LIME to explain predictions from the XGBoost model. LIME explains one prediction at a time by showing which feature ranges pushed the prediction toward benign or attack. This is useful because a security analyst can review the main reasons for a prediction instead of only seeing the final class label.

6.1 Attack Flow Explanation

For one attack example, the actual label was Attack and the predicted probability of attack was 1.0000. The LIME explanation showed that several timing and packet features influenced the prediction. Some features pushed the prediction toward attack, while others pushed it away. This is expected because LIME explains a local decision using weighted feature intervals.

```
Actual label: Attack
Predicted probability of attack: 1.0000

Top features influencing this prediction:
  FIN Flag Count <= 0.00                                    weight = -0.0743
  20.00 < min_seg_size_forward <= 32.00                     weight = +0.0427
  Fwd IAT Max > 2141921.25                                  weight = +0.0241
  Total Fwd Packets > 5.00                                  weight = -0.0240
  Bwd Packet Length Std > 149.53                            weight = +0.0211
  Fwd PSH Flags <= 0.00                                     weight = +0.0196
  Bwd Header Length > 112.00                                weight = -0.0179
  Packet Length Std > 240.81                                weight = +0.0169
  Fwd IAT Mean > 536123.89                                  weight = +0.0136
  RST Flag Count <= 0.00                                    weight = +0.0004
```

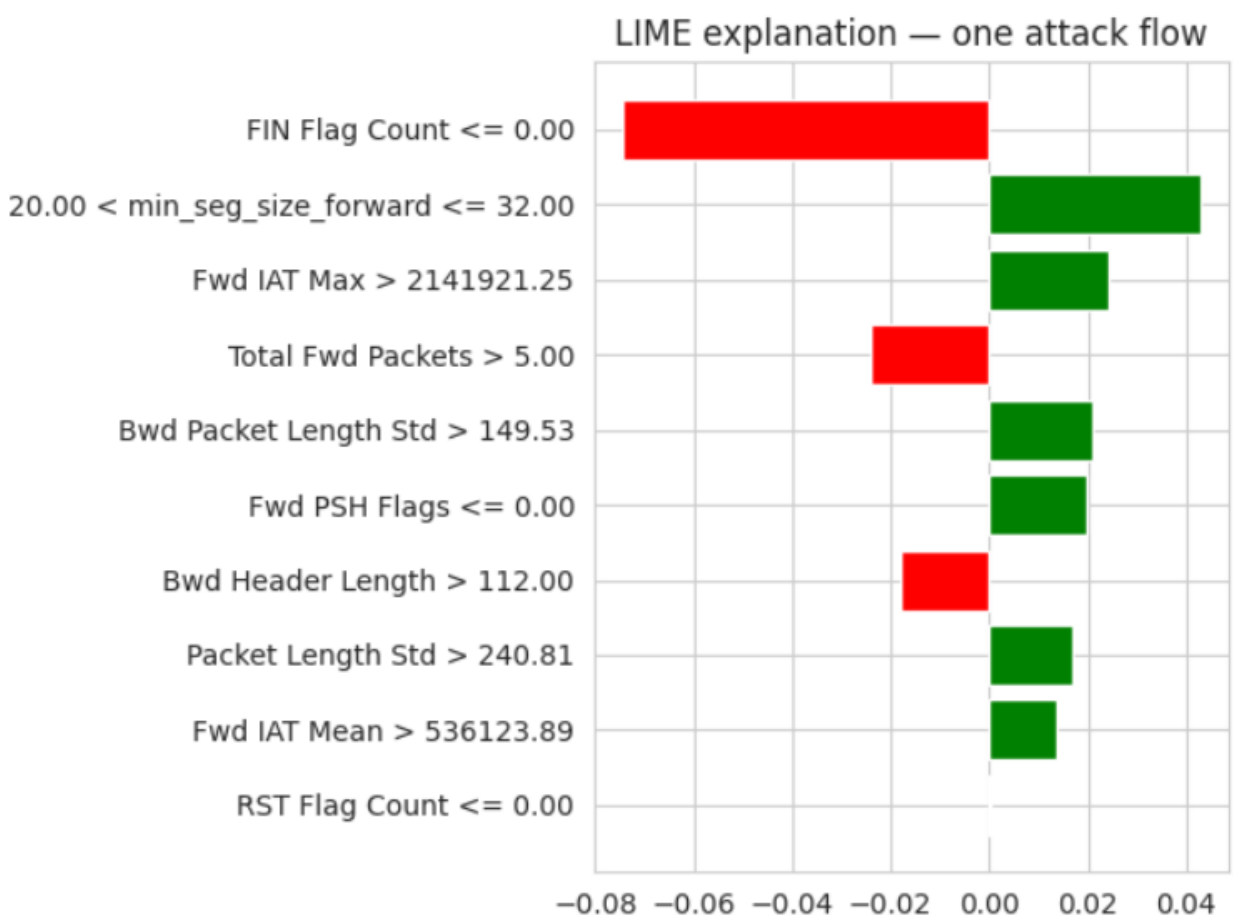


Figure 11. LIME explanation for one attack flow.

6.2 Benign Flow Explanation

For one benign example, the actual label was Benign and the predicted probability of attack was 0.0003. The strongest negative weights pushed the model away from the attack class. This local explanation helps confirm that the model is not only accurate but also produces a reviewable reason for individual predictions.

```
Actual label: Benign
Predicted probability of attack: 0.0016

Top features influencing this prediction:
  FIN Flag Count <= 0.00                                  weight = -0.0527
  min_seg_size_forward <= 20.00                           weight = -0.0396
  Destination Port > 443.00                               weight = -0.0373
  Total Length of Bwd Packets <= 6.00                     weight = +0.0261
  251.00 < Init_Win_bytes_forward <= 8192.00              weight = +0.0219
  64.00 < Fwd Header Length <= 132.00                     weight = +0.0156
  Idle Min <= 0.00                                        weight = -0.0139
  Active Min <= 0.00                                      weight = +0.0129
  Total Backward Packets <= 1.00                          weight = +0.0123
  ECE Flag Count <= 0.00                                  weight = +0.0023
```

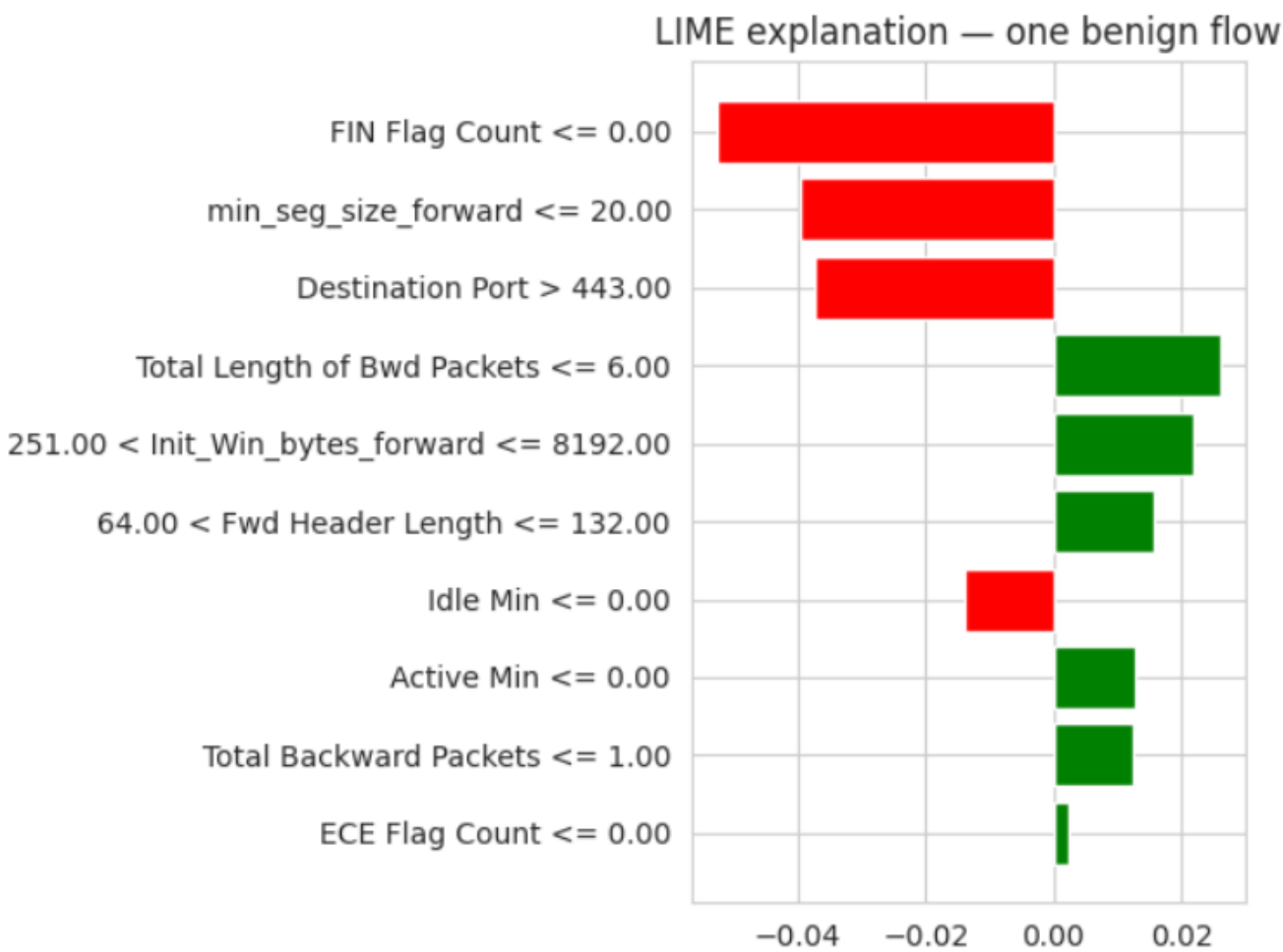


Figure 12. LIME explanation for one benign flow.

7. Fairness Audit

A traditional fairness audit was not applicable to this project because the dataset contains network-flow features, not human protected attributes such as race, gender, age, religion, or disability. The model is not making decisions about people; it is classifying network flows. For this project, the more relevant ethical and operational concerns are false positives, false negatives, and explainability.

False negatives are important because they represent attacks that the model missed. False positives are also important because too many incorrect alerts can waste analyst time. The final XGBoost model had only five false negatives on the test set, which was the best result among the tested models. The use of LIME also improves transparency because it gives a short explanation for individual predictions.

8. Conclusion and Future Work

This project showed that supervised machine learning can be used to classify CIC-IDS2017 network flows as benign or attack traffic. The data was cleaned, checked for duplicates, checked for missing and infinite values, encoded into binary labels, and split into training and test sets. Three models were trained and compared. Logistic Regression provided a useful baseline, Random Forest achieved very strong results, and XGBoost performed best overall.

The best model was XGBoost, which achieved 0.9991 accuracy, 0.9955 precision, 0.9995 recall, 0.9975 F1-score, and 1.0000 ROC-AUC. The confusion matrix showed that XGBoost missed only five attack flows in the test set. This is important because missed attacks are one of the most serious problems in intrusion detection. The LIME explanations also showed how individual predictions could be connected back to traffic features, which makes the model more understandable.

Future work could improve the project in several ways:

- Use the full CIC-IDS2017 dataset on a machine with more memory instead of a 10% chunk sample.
- Try multi-class classification so the model predicts the exact attack type, not just attack vs. benign.
- Validate the model on a different intrusion detection dataset to check whether it generalizes.
- Compare LIME with SHAP to see which explanation method gives more stable feature explanations.
- Build a simple dashboard or alerting interface that shows the predicted class, probability, and explanation for each suspicious flow.

References

[1] Canadian Institute for Cybersecurity, University of New Brunswick. "Intrusion Detection Evaluation Dataset (CIC-IDS2017)." https://www.unb.ca/cic/datasets/ids-2017.html

[2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization." Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108-116. DOI: 10.5220/0006639801080116.

[3] A. H. Lashkari, “CICFlowMeter,” GitHub. Available: https://github.com/ahlashkari/CICFlowMeter

[4] J. Wilkie, H. Hindy, C. Tachtatzis, and R. Atkinson, “Contrastive Self-Supervised Network Intrusion Detection using Augmented Negative Pairs,” 2025. Available: https://huggingface.co/papers/2509.06550

[5] M. T. Ribeiro, S. Singh, and C. Guestrin. "Why Should I Trust You?: Explaining the Predictions of Any Classifier." Proceedings of NAACL-HLT Demonstrations, 2016, pp. 97-101.

Contributions

Team Member	Contributions
Damini	Project proposal, dataset selection, data loading, preprocessing, exploratory data analysis, model training, hyperparameter tuning, evaluation, LIME explainability, report writing, and presentation preparation.
CHATGPT	Used for report formatting and Help with LIME in code.